



Sichuan University -  
Pittsburgh Institute

# ECEo202 Embedded Processors & Interfacing + Lab

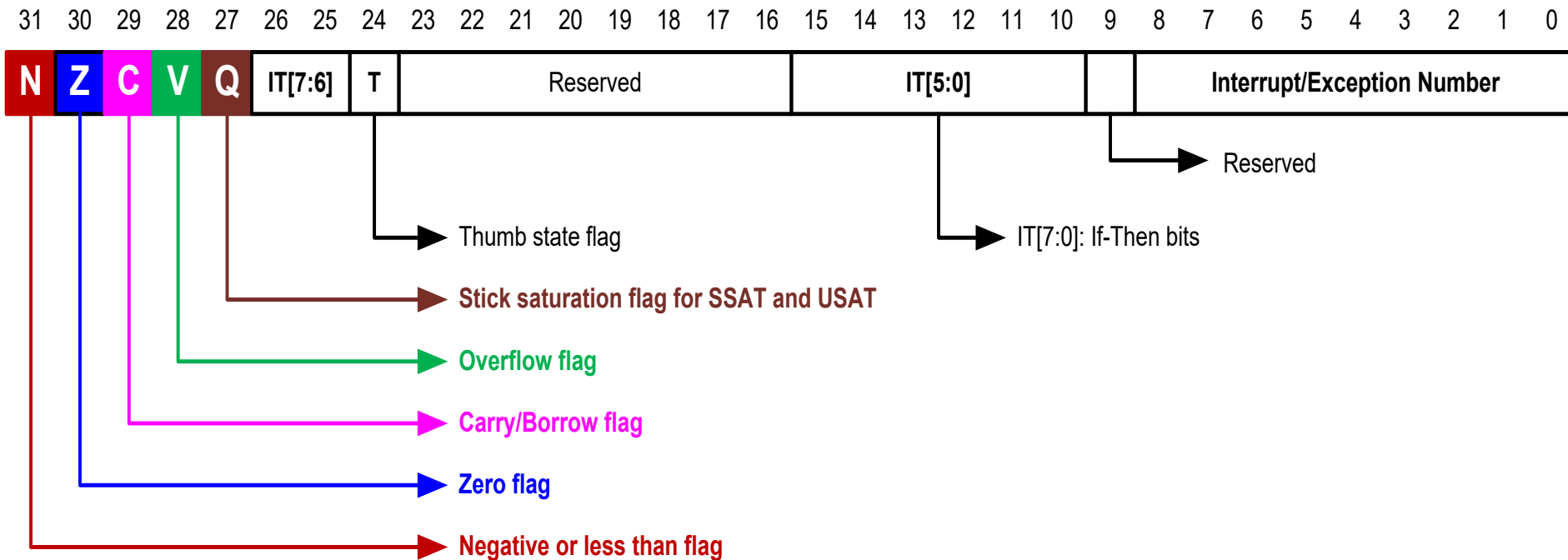
Sichuan University-Pittsburgh Institute



# Overview

- Comparison Instruction
- Branch and Condition Codes
- If-then-else
- While loop
- For loop

# Combined Program Status Registers (xPSR)



# Comparison Instructions

| Instruction | Operands | Brief description | Flags   |
|-------------|----------|-------------------|---------|
| <b>CMP</b>  | Rn, Op2  | Compare           | N,Z,C,V |
| <b>CMN</b>  | Rn, Op2  | Compare Negative  | N,Z,C,V |
| <b>TEQ</b>  | Rn, Op2  | Test Equivalence  | N,Z,C   |
| <b>TST</b>  | Rn, Op2  | Test              | N,Z,C   |

- The only effect of the comparisons is to **update the condition flags**.  
No need to set S bit. No need to specify Rd.

- Operations are:

- **CMP** operand1 - operand2, but result not written
- **CMN** operand1 + operand2, but result not written
- **TST** operand1 & operand2, but result not written
- **TEQ** operand1 ^ operand2, but result not written

- Examples:

**CMP** r0, r1

**TST** r2, #5

# CMP and CMN

**CMP**{cond} Rn, Operand2

**CMN**{cond} Rn, Operand2

- The CMP instruction **subtracts** the value of Operand2 from the value in Rn.
  - This is the same as a SUBS instruction, except that the result is discarded.
- The CMN instruction **adds** the value of Operand2 to the value in Rn.
  - This is the same as an ADDS instruction, except that the result is discarded.
- These instructions update the N, Z, C and V flags according to the result.

# Example of CMP

$$f(x) = |x|$$

```
Area absolute, CODE, READONLY
EXPORT __main
ENTRY

__main PROC
    CMP     r1, #0
    RSBLT   r1, r1, #0 ; Reverse Subtract if Less Than
done      B done      ; deadlock

    ENDP
END
```

*Note: RSB = Reverse SuBtract*

# TST and TEQ

**TST**{cond} Rn, Operand2 ; Bitwise AND

**TEQ**{cond} Rn, Operand2 ; Bitwise Exclusive OR

- The TST instruction performs a **bitwise AND** operation on the value in Rn and the value of Operand2.
  - This is the same as a ANDS instruction, except that the result is discarded.
- The TEQ instruction performs a **bitwise Exclusive OR (XOR)** operation on the value in Rn and the value of Operand2.
  - This is the same as a EORS instruction, except that the result is discarded.
- **Update the N and Z flags** according to the result
- Can update the C flag during the calculation of Operand2
- **Do not affect the V flag**

# Branch Instructions

| Instruction | Operands | Brief description         | Flags |
|-------------|----------|---------------------------|-------|
| <b>B</b>    | label    | Branch                    | -     |
| <b>BL</b>   | label    | Branch with Link          | -     |
| <b>BX</b>   | Rm       | Branch indirect           | -     |
| <b>BLX</b>  | Rm       | Branch indirect with Link | -     |

- **B label**: causes a branch to label.
- **BL label**: instruction copies the address of the next instruction into r14 (lr, the link register), and causes a branch to label.
- **BX Rm**: branch to the address held in Rm
- **BLX Rm**: copies the address of the next instruction into r14 (lr, the link register) and branch to the address held in Rm



# Condition Codes

| Suffix | Description               | Flags tested |
|--------|---------------------------|--------------|
| EQ     | EQual                     |              |
| NE     | Not EQual                 |              |
| CS/HS  | Unsigned Higher or Same   |              |
| CC/LO  | Unsigned LOwer            |              |
| MI     | MInus (Negative)          |              |
| PL     | PLus (Positive or Zero)   |              |
| VS     | oVerflow Set              |              |
| VC     | oVerflow Clear            |              |
| HI     | Unsigned HIgher           |              |
| LS     | Unsigned Lower or Same    |              |
| GE     | Signed Greater or Equal   |              |
| LT     | Signed Less Than          |              |
| GT     | Signed Greater Than       |              |
| LE     | Signed Less than or Equal |              |
| AL     | ALways                    |              |

*Note AL is the default and does not need to be specified*

# Condition Codes

- The possible condition codes are listed below:

| Suffix | Description               | Flags tested |
|--------|---------------------------|--------------|
| EQ     | EQual                     | Z=1          |
| NE     | Not EQual                 | Z=0          |
| CS/HS  | Unsigned Higher or Same   | C=1          |
| CC/LO  | Unsigned LOwer            | C=0          |
| MI     | MInus (Negative)          | N=1          |
| PL     | PLus (Positive or Zero)   | N=0          |
| VS     | OVerflow Set              | V=1          |
| VC     | oVerflow Clear            | V=0          |
| HI     | Unsigned HIgher           | C=1 & Z=0    |
| LS     | Unsigned Lower or Same    | C=0 or Z=1   |
| GE     | Signed Greater or Equal   | N=V          |
| LT     | Signed Less Than          | N!=V         |
| GT     | Signed Greater Than       | Z=0 & N=V    |
| LE     | Signed Less than or Equal | Z=1 or N!=V  |
| AL     | ALways                    |              |

*Note AL is the default and does not need to be specified*

# Signed vs. Unsigned

Conditional codes applied to branch instructions

| Compare | Signed | Unsigned |
|---------|--------|----------|
| ==      | EQ     | EQ       |
| ≠       | NE     | NE       |
| >       | GT     | HI       |
| ≥       | GE     | HS       |
| <       | LT     | LO       |
| ≤       | LE     | LS       |



| Compare | Signed | Unsigned |
|---------|--------|----------|
| ==      | BEQ    | BEQ      |
| !=      | BNE    | BNE      |
| >       | BGT    | BHI      |
| >=      | BGE    | BHS      |
| <       | BLT    | BLO      |
| <=      | BLE    | BLS      |

# Branch Instructions

|                             | Instruction   | Description                         | Flags tested    |
|-----------------------------|---------------|-------------------------------------|-----------------|
| <b>Unconditional Branch</b> | B label       | Branch to label                     |                 |
| <b>Conditional Branch</b>   | BEQ label     | Branch if Equal                     | Z = 1           |
|                             | BNE label     | Branch if Not Equal                 | Z = 0           |
|                             | BCS/BHS label | Branch if unsigned Higher or Same   | C = 1           |
|                             | BCC/BLO label | Branch if unsigned Lower            | C = 0           |
|                             | BMI label     | Branch if MINus (Negative)          | N = 1           |
|                             | BPL label     | Branch if PPlus (Positive or Zero)  | N = 0           |
|                             | BVS label     | Branch if oVerflow Set              | V = 1           |
|                             | BVC label     | Branch if oVerflow Clear            | V = 0           |
|                             | BHI label     | Branch if unsigned HIgher           | C = 1 & Z = 0   |
|                             | BLS label     | Branch if unsigned Lower or Same    | C = 0 or Z = 1  |
|                             | BGE label     | Branch if signed Greater or Equal   | N = V           |
|                             | BLT label     | Branch if signed Less Than          | N != V          |
|                             | BGT label     | Branch if signed Greater Than       | Z = 0 & N = V   |
|                             | BLE label     | Branch if signed Less than or Equal | Z = 1 or N = !V |

# Number Interpretation

Which is greater?

**0xFFFFFFFF** or **0x00000001**

- If they represent signed numbers, the latter is greater  
(**1 > -1**).
- If they represent unsigned numbers, the former is greater  
(**4294967295 > 1**).

# Which is Greater: 0xFFFFFFFF or 0x00000001?

It's **software's responsibility** to tell computer how to interpret data:

- If written in C, declare the signed vs unsigned variable
- If written in Assembly, use signed vs unsigned branch instructions

```
signed int x, y ;  
x = -1;  
y = 1;  
if (x > y)  
    ...
```

```
MOVS r6, #0xFFFFFFFF  
MOVS r5, #0x00000001  
CMP  r5, r6  
BLE Then_Clause  
...
```

**BLE:** Branch if less than or equal, signed  $\leq$

```
unsigned int x, y ;  
x = 4294967295;  
y = 1;  
if (x > y)  
    ...
```

```
MOVS r6, #0xFFFFFFFF  
MOVS r5, #0x00000001  
CMP  r5, r6  
BLS Then_Clause  
...
```

**BLS:** Branch if lower or same, unsigned  $\leq$

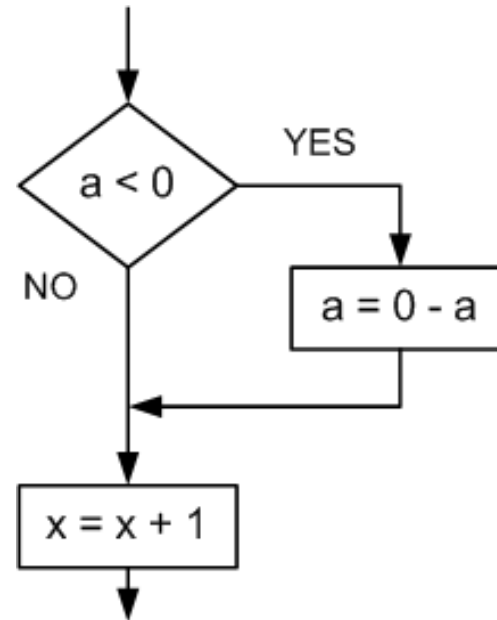


# How to implement C Branches?

# If-then Statement

## C Program

```
if (a < 0) {  
    a = 0 - a;  
}  
x = x + 1;
```



## Implementation 1:

; assume r1 = a, r2 = x



# Compound Boolean Expression

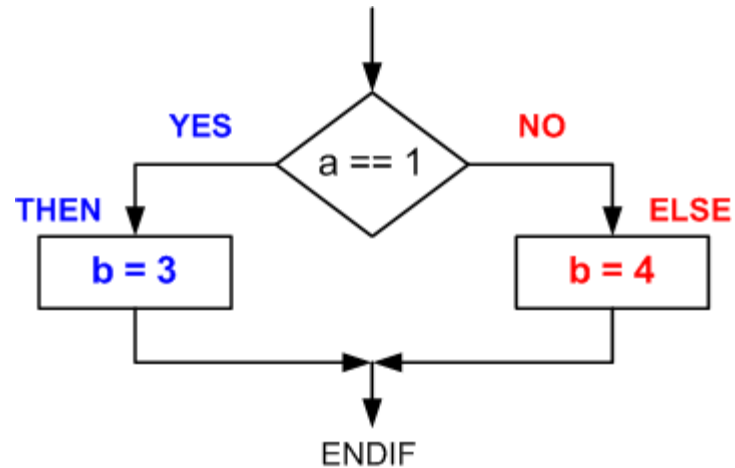
`x > 20 && x < 25`  
`x == 20 || x == 25`  
`!(x == 20 || x == 25)`

| C Program                                                                      | Assembly Program            |
|--------------------------------------------------------------------------------|-----------------------------|
| <pre>// x is a signed integer  if(x &lt;= 20    x &gt;= 25){     a = 1 }</pre> | <pre>; r0 = x, r1 = a</pre> |

# If-then-else

## C Program

```
if (a == 1)
    b = 3;
else
    b = 4;
```



```
; r1 = a, r2 = b
```

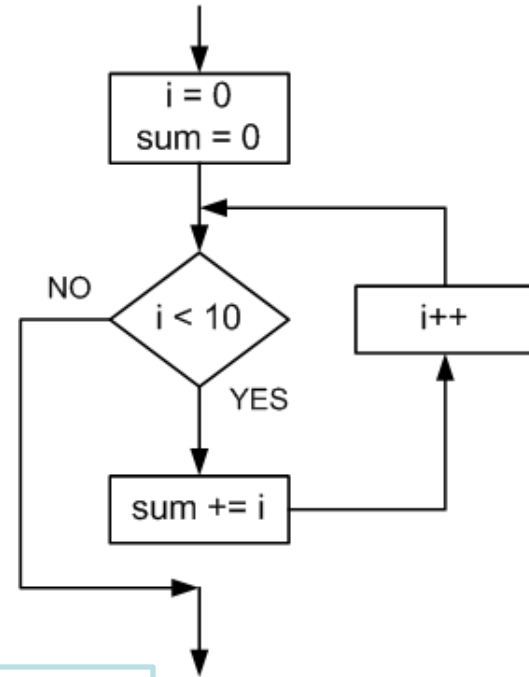
# For Loop

## C Program

```
int i;  
int sum = 0;  
for(i = 0; i < 10; i++){  
    sum += i;  
}
```

Implementation :

```
MOV r0, #0    ; i  
MOV r1, #0    ; sum
```





# Condition Execution

# Condition Codes

- The possible condition codes are listed below:

| Suffix       | Description               | Flags tested |
|--------------|---------------------------|--------------|
| <b>EQ</b>    | Equal                     | Z=1          |
| <b>NE</b>    | Not equal                 | Z=0          |
| <b>CS/HS</b> | Unsigned higher or same   | C=1          |
| <b>CC/LO</b> | Unsigned lower            | C=0          |
| <b>MI</b>    | Negative                  | N=1          |
| <b>PL</b>    | Positive or Zero          | N=0          |
| <b>VS</b>    | Overflow                  | V=1          |
| <b>VC</b>    | No overflow               | V=0          |
| <b>HI</b>    | Unsigned higher           | C=1 & Z=0    |
| <b>LS</b>    | Unsigned lower or same    | C=0 or Z=1   |
| <b>GE</b>    | Signed Greater or equal   | N=V          |
| <b>LT</b>    | Signed Less than          | N!=V         |
| <b>GT</b>    | Signed Greater than       | Z=0 & N=V    |
| <b>LE</b>    | Signed Less than or equal | Z=1 or N!=V  |
| <b>AL</b>    | Always                    |              |

*Note AL is the default and does not need to be specified*

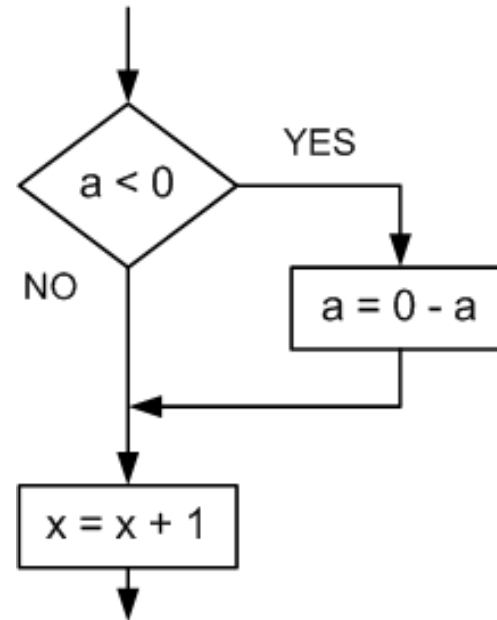
# Conditional Execution

| Add instruction         | Condition                        | Flag tested            |
|-------------------------|----------------------------------|------------------------|
| <b>ADDEQ</b> r3, r2, r1 | Add if Equal                     | Add if Z = 1           |
| <b>ADDNE</b> r3, r2, r1 | Add if Not Equal                 | Add if Z = 0           |
| <b>ADDHS</b> r3, r2, r1 | Add if Unsigned Higher or Same   | Add if C = 1           |
| <b>ADDLO</b> r3, r2, r1 | Add if Unsigned LOwer            | Add if C = 0           |
| <b>ADDMI</b> r3, r2, r1 | Add if Minus (Negative)          | Add if N = 1           |
| <b>ADDPL</b> r3, r2, r1 | Add if PLus (Positive or Zero)   | Add if N = 0           |
| <b>ADDVS</b> r3, r2, r1 | Add if oVerflow Set              | Add if V = 1           |
| <b>ADDVC</b> r3, r2, r1 | Add if oVerflow Clear            | Add if V = 0           |
| <b>ADDHI</b> r3, r2, r1 | Add if Unsigned HIgher           | Add if C = 1 & Z = 0   |
| <b>ADDLS</b> r3, r2, r1 | Add if Unsigned Lower or Same    | Add if C = 0 or Z = 1  |
| <b>ADDGE</b> r3, r2, r1 | Add if Signed Greater or Equal   | Add if N = V           |
| <b>ADDLT</b> r3, r2, r1 | Add if Signed Less Than          | Add if N != V          |
| <b>ADDGT</b> r3, r2, r1 | Add if Signed Greater Than       | Add if Z = 0 & N = V   |
| <b>ADDLE</b> r3, r2, r1 | Add if Signed Less than or Equal | Add if Z = 1 or N = !V |

# If-then Statement

## C Program

```
if (a < 0) {
    a = 0 - a;
}
x = x + 1;
```



## Implementation 1:

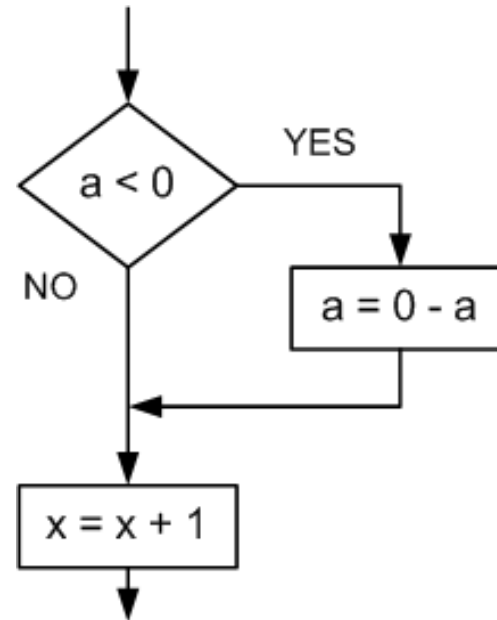
```

; assume r1 = a, r2 = x
CMP r1, #0           ; Compare a with 0
BGE endif          ; Go to endif if a ≥ 0
then  RSB r1, r1, #0  ; a = - a
endif  ADD r2, r2, #1 ; x = x + 1
```

# If-then Statement

## C Program

```
if (a < 0) {  
    a = 0 - a;  
}  
x = x + 1;
```



## Implementation 2:

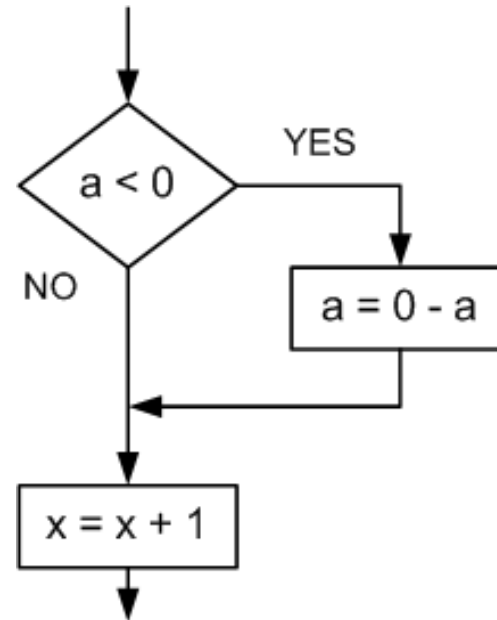
```
; r1 = a, r2 = x  
CMP    r1, #0      ; Compare a with 0
```



# If-then Statement

## C Program

```
if (a < 0) {
    a = 0 - a;
}
x = x + 1;
```



## Implementation 2:

```

; r1 = a, r2 = x
CMP    r1, #0      ; Compare a with 0
RSBLT r1, r1, #0  ; a = 0 - a if a < 0
ADD    r2, r2, #1  ; x = x + 1
```

# Example of Conditional Execution

```
if (a <= 0)
    y = -1;
else
    y = 1;
```



a → r0

y → r1

```
CMP    r0, #0
```

LE: Signed Less than or Equal  
GT: Signed Greater Than

# Example of Conditional Execution

```
if (a==1 || a==7 || a==11)
    y = 1;
else
    y = -1;
```



$a \rightarrow r0$

$y \rightarrow r1$

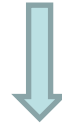
```
CMP    r0, #1
```

NE: Not Equal  
EQ: Equal

# Example 1: Greatest Common Divider (GCD)

## Euclid's Algorithm

```
While (a != b) {  
    if (a > b) a = a - b;  
    else b = b - a;  
}
```



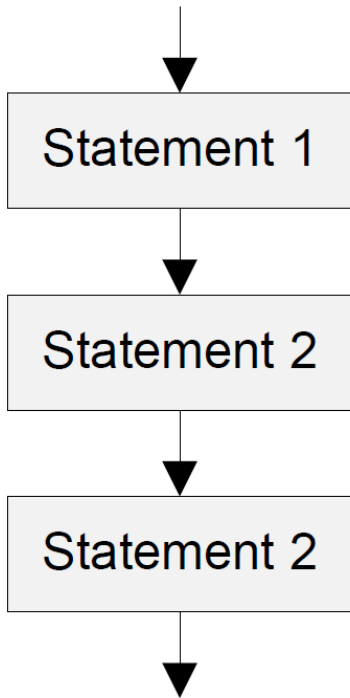
```
gcd      ; suppose r0 = a and r1 = b  
CMP r0, r1      ; a > b?  
BEQ end        ; if a = b, done  
  
BLT less      ; a < b  
SUB r0, r0, r1  ; a = a - b  
B gcd  
  
less     SUB r1, r1, r0  ; b = b - a  
B gcd  
  
end
```

# Divide and Conquer

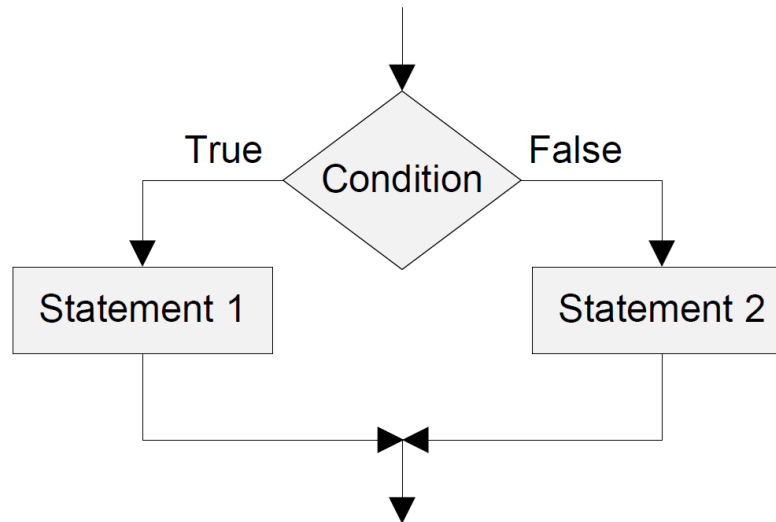
*"Nothing is particularly hard if you divide it into small jobs."*

Henry Ford, Founder of Ford Motor

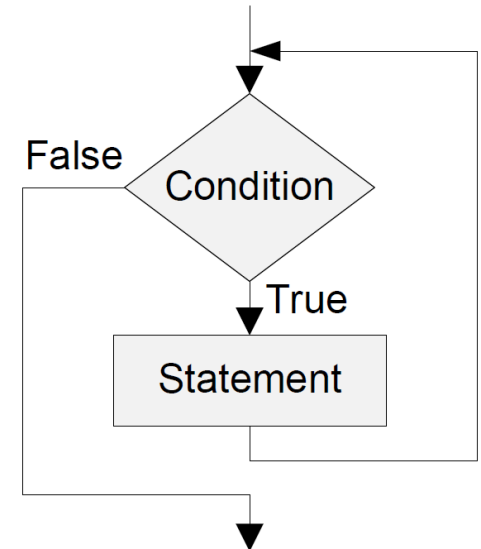
# Three basic control structures



Sequence  
Structure



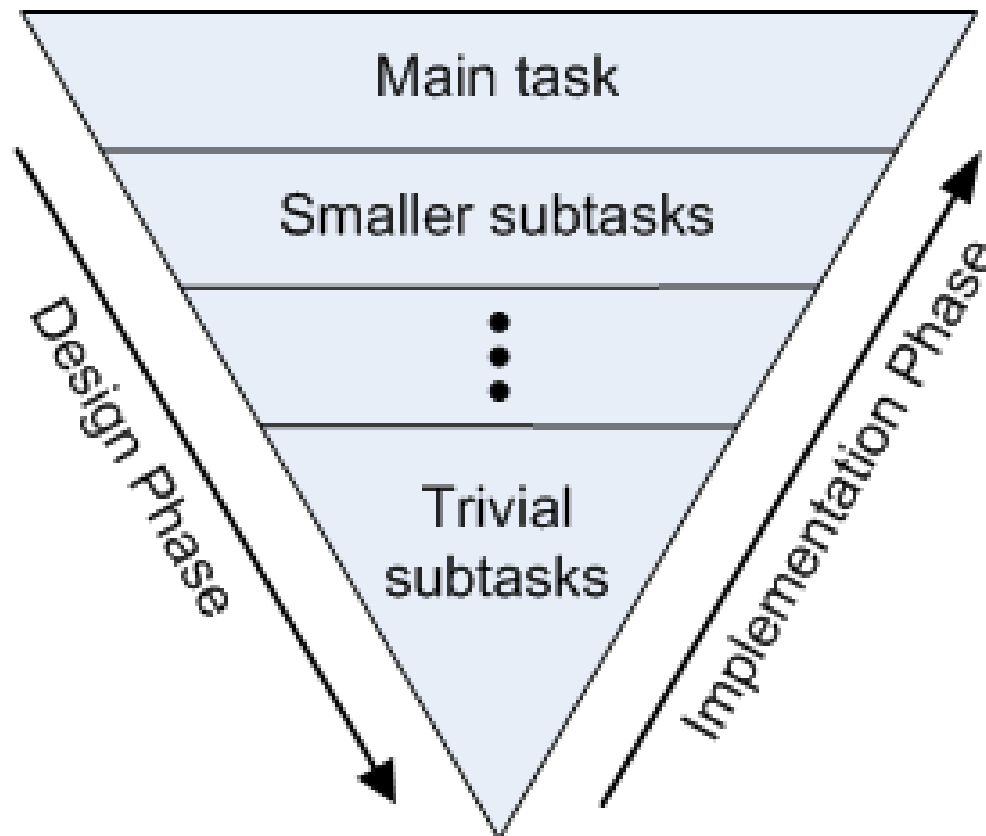
Selection Structure



Loop Structure

# Top-Down Design

## Top-Down Design

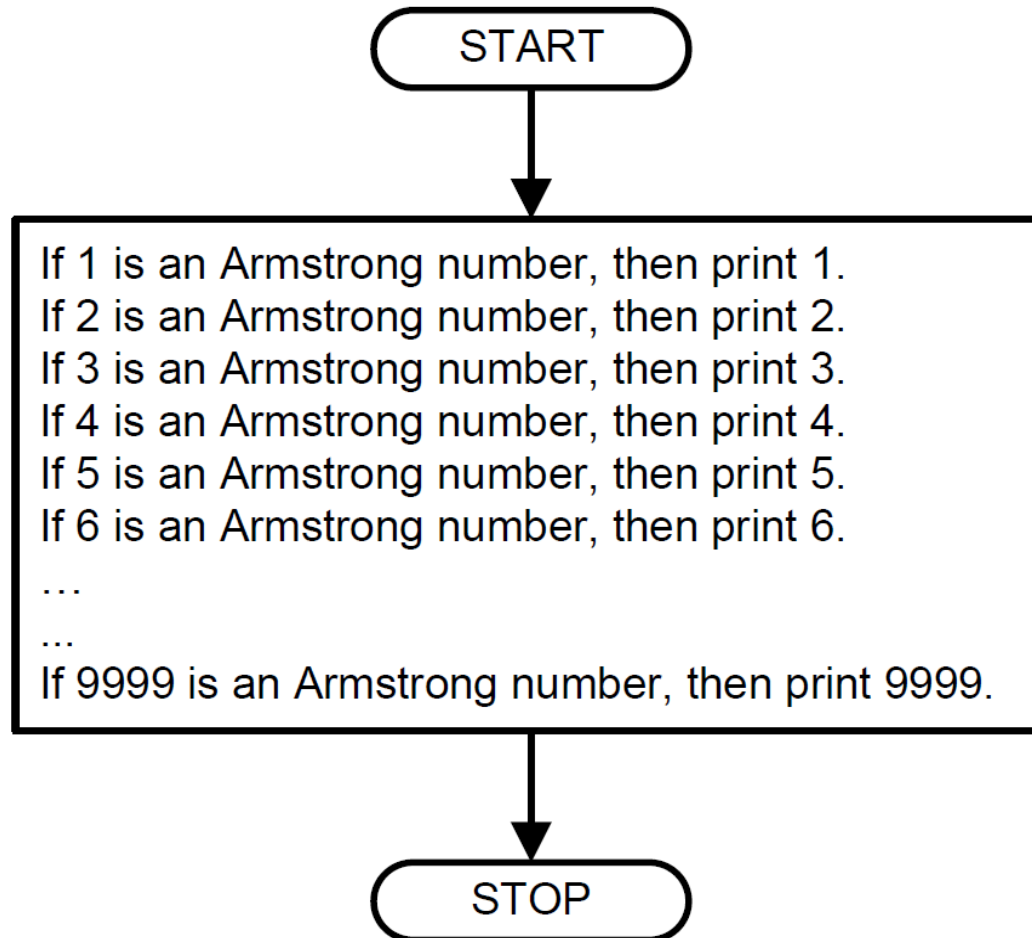


# Top-Down Design Example

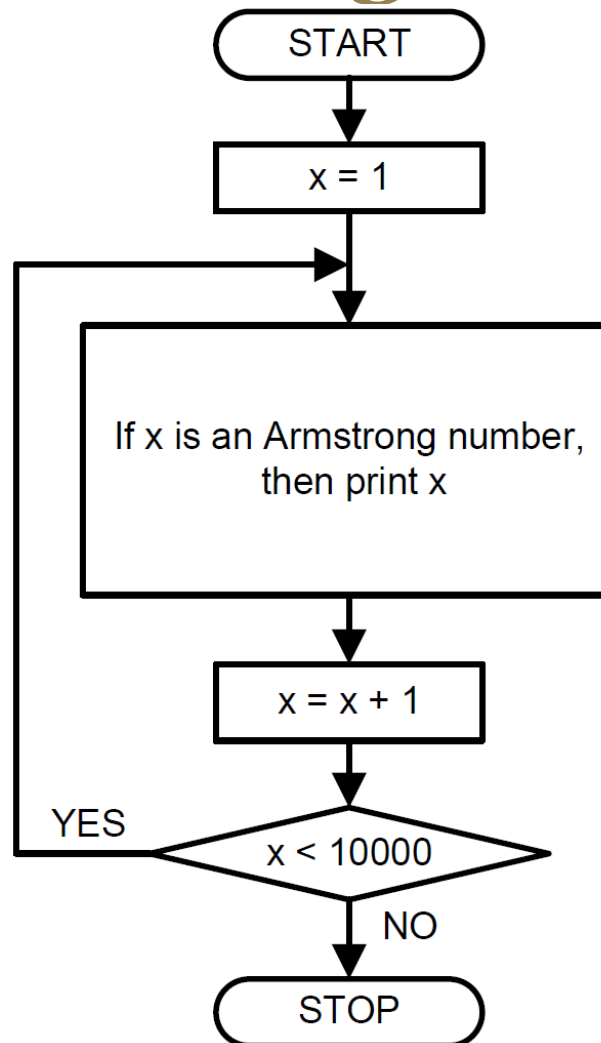
- Find all Armstrong numbers less than 10,000
- Given a positive integer that has  $n$  digits, it is an Armstrong number if the sum of the  $n^{\text{th}}$  powers of its digits equals the number itself.
- For example, 371 is an Armstrong number since we have  $371 = 3^3 + 7^3 + 1^3$ .



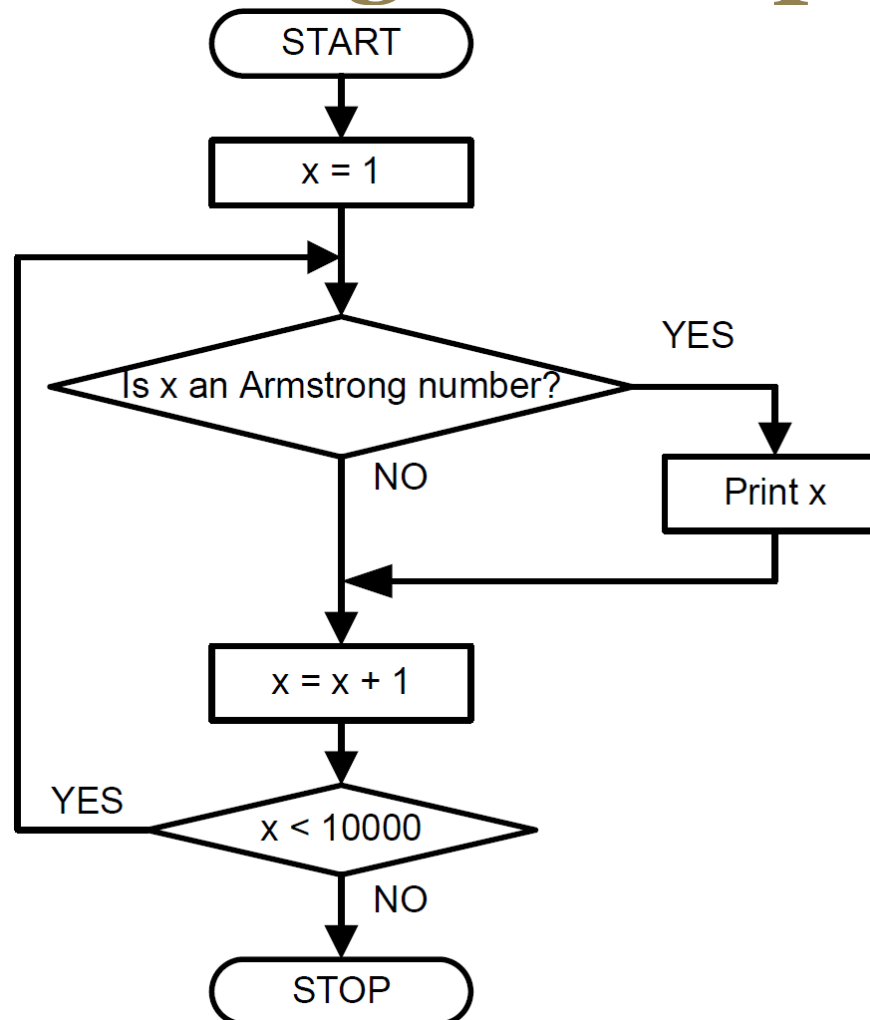
# Top-Down Design Example



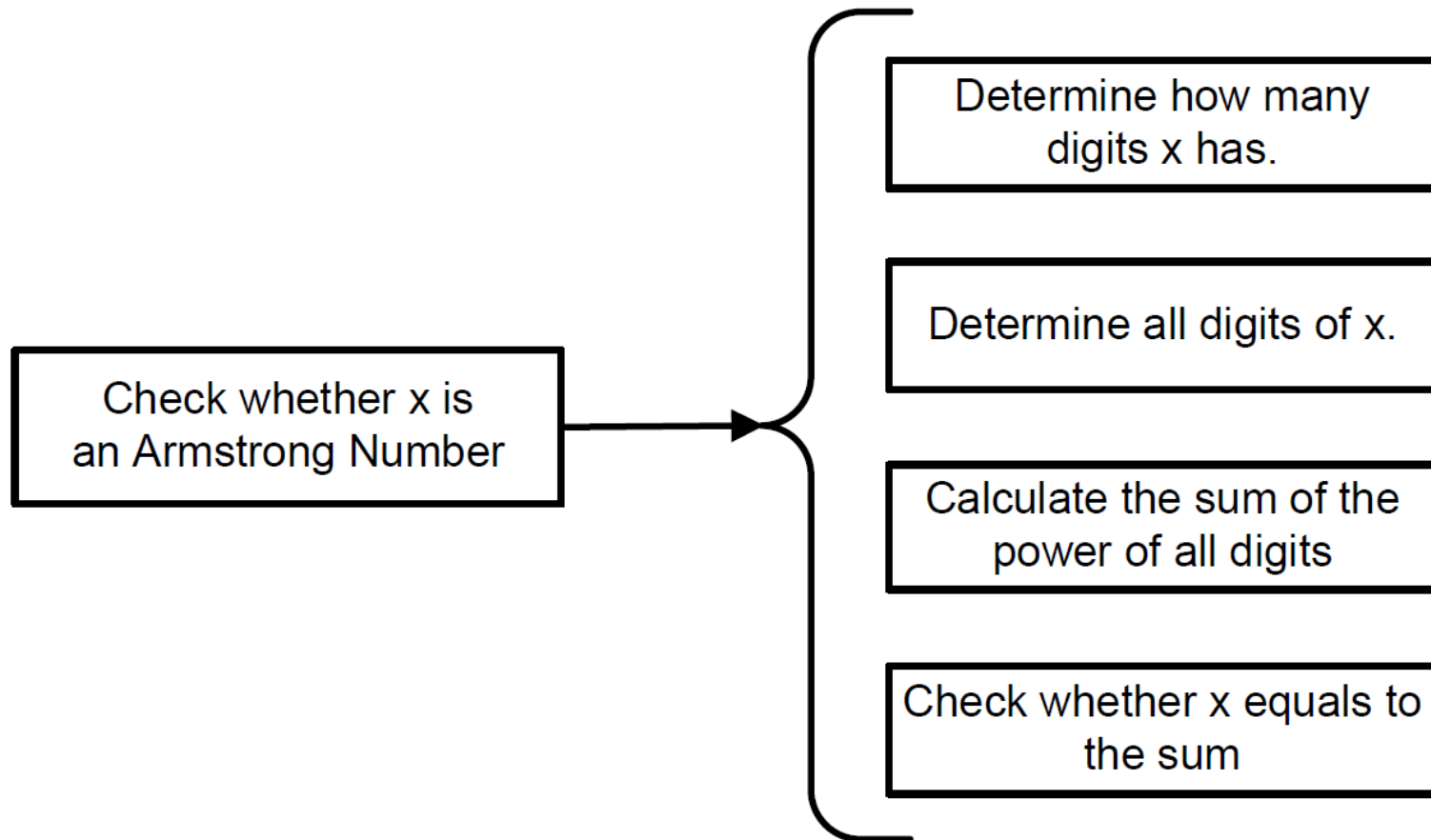
# Top-Down Design Example



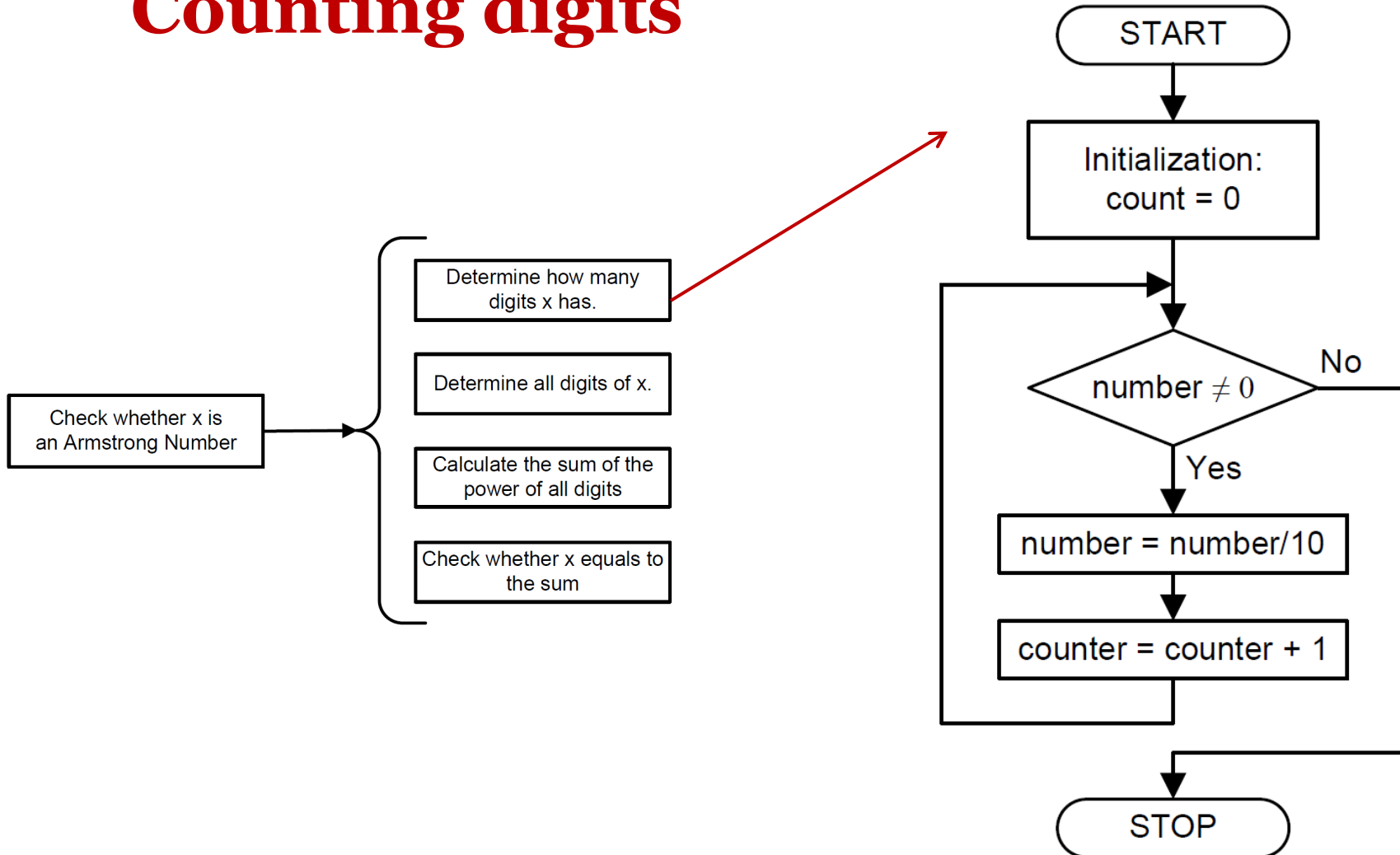
# Top-Down Design Example



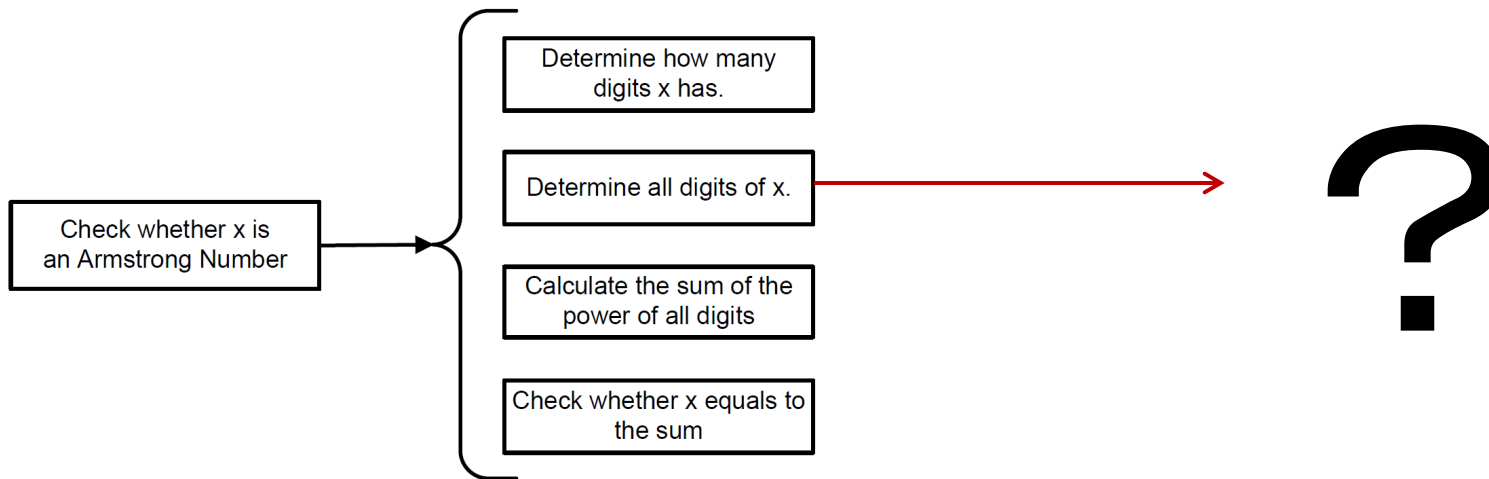
# Top-Down Design Example



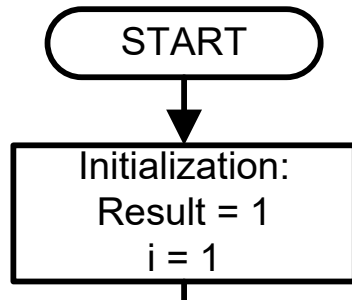
# Top-Down Design Example: Counting digits



# Top-Down Design Example: Counting digits

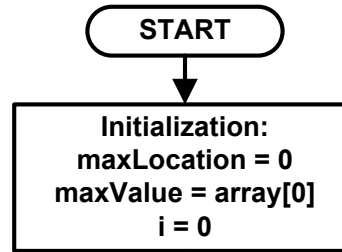


# Factorial Numbers



$$n! = \prod_{i=1}^n i = n \times (n - 1) \times (n - 2) \cdots \times 2 \times 1$$

# Finding Max of an Array



Assume we know  
the array size



# Reuse Registers

## Data memory

```
int A = 0; // 0x00000000
int B = -1; // 0xFFFFFFFF
int C = -2; // 0xFFFFFFF
int D = 2; // 0x00000002

void main(void){
    A = B + C - D;
    return;
}
```

| Address     | Data |                      |
|-------------|------|----------------------|
| 0x2000,000F | 0x00 | D = 0x0000,0002 = 2  |
| 0x2000,000E | 0x00 |                      |
| 0x2000,000D | 0x00 |                      |
| 0x2000,000C | 0x02 |                      |
| 0x2000,000B | 0xFF | C = 0xFFFF,FFF = -2  |
| 0x2000,000A | 0xFF |                      |
| 0x2000,0009 | 0xFF |                      |
| 0x2000,0008 | 0xFE |                      |
| 0x2000,0007 | 0xFF | B = 0xFFFF,FFFF = -1 |
| 0x2000,0006 | 0xFF |                      |
| 0x2000,0005 | 0xFF |                      |
| 0x2000,0004 | 0xFF |                      |
| 0x2000,0003 | 0x00 | A = 0x0000,0000 = 0  |
| 0x2000,0002 | 0x00 |                      |
| 0x2000,0001 | 0x00 |                      |
| 0x2000,0000 | 0x00 |                      |

# Reuse Registers

```
int A = 0; // 0x00000000
int B = -1; // 0xFFFFFFFF
int C = -2; // 0xFFFFFFFFE
int D = 2; // 0x00000002

void main(void){
    A = B + C - D;
    return;
}
```

How many registers  
are used?

```
AREA myCode, CODE
```

```
EXPORT __main
```

```
ENTRY
```

```
__main PROC
```

```
LDR r2, =B ; r2 = 0x2000,0004
```

```
LDR r3, [r2] ; r3 = B = -1
```

```
LDR r4, =C ; r4 = 0x2000,0008
```

```
LDR r5, [r4] ; r5 = C = -2
```

```
LDR r6, =D ; r6 = 0x2000,000B
```

```
LDR r7, [r6] ; r7 = D = 2
```

```
ADD r1, r3, r5 ; r1 = B + C
```

```
SUB r1, r1, r7 ; r1 = B + C - D
```

```
LDR r0, =A ; r0 = 0x2000,0000
```

```
STR r1, [r0] ; Save A
```

```
ENDP
```

```
AREA myData, DATA
```

```
A DCD 0
```

```
B DCD -1
```

```
C DCD -2
```

```
D DCD 2
```

```
END
```

# Reuse Registers

```
AREA myCode, CODE
EXPORT __main
ENTRY
__main PROC
```

*Lifetime of r3*

```
LDR r2, =B
LDR r3, [r2]
LDR r4, =C
LDR r5, [r4]
LDR r6, =D
LDR r7, [r6]
ADD r1, r3, r5
SUB r1, r1, r7
LDR r0, =A
STR r1, [r0]
ENDP
```

```
AREA myData, DATA
A DCD 0
B DCD -1
C DCD -2
D DCD 2

END
```

8 registers used

```
AREA myCode, CODE
EXPORT __main
ENTRY
__main PROC
```

*Lifetime of r2*

```
LDR r2, =B
LDR r3, [r2]
LDR r2, =C
LDR r5, [r2]
LDR r2, =D
LDR r7, [r2]
ADD r3, r3, r5
SUB r3, r3, r7
LDR r2, =A
STR r3, [r2]
ENDP
```

*Lifetime of r2*

```
AREA myData, DATA
A DCD 0
B DCD -1
C DCD -2
D DCD 2

END
```

4 registers used

```
AREA myCode, CODE
EXPORT __main
ENTRY
__main PROC
```

```
LDR r2, =B
LDR r3, [r2]
LDR r2, =C
LDR r5, [r2]
LDR r2, =D
LDR r2, [r2]
ADD r3, r3, r5
SUB r3, r3, r2
LDR r2, =A
STR r3, [r2]
ENDP
```

*Reuse r2*

```
AREA myData, DATA
A DCD 0
B DCD -1
C DCD -2
D DCD 2

END
```

3 registers used